

Exercice 1.

Pour la suite d'instructions ci-contre, donner instruction par instruction les valeurs des registres `esp` et `eax` ainsi que l'état de la pile.

```
mov eax, 0xABCDEF01
```

=> `eax = 0xABCDEF01`, `esp = X`, pile : (rien de défini au-dessus de X)

```
push eax
```

=> `esp` diminue de 4 → `esp = X - 4`, `[X-4] = 0xABCDEF01`, `eax = 0xABCDEF01`, Pile (du bas vers le haut) : à `X-4` : `0xABCDEF01`, `esp` pointe sur `X-4`.

```
mov eax, 0x01234567
```

=> `eax = 0x01234567`, `esp = X - 4`, pile inchangée : `[X-4] = 0xABCDEF01`

```
push eax
```

=> `esp` diminue de 4 → `esp = X - 8`, `[X-8] = 0x01234567`, pile : `X-8` : `0x01234567` (sommet), `X-4` : `0xABCDEF01`

```
pop eax
```

=> `eax` reçoit la valeur au sommet de pile → `eax = [X-8] = 0x01234567`, `esp` augmente de 4 → `esp = X - 4`, pile (logiquement) : sommet à `X-4` : `0xABCDEF01`

```
pop eax
```

=> `eax = [X-4] = 0xABCDEF01`, `esp` augmente de 4 → `esp = X` (pile comme au début)

=> `eax` contient la val `0xABCDEF01` et `esp` ne change pas car on a pas modifié la pile (donc vaut x)

=> mtn `esp` vaut l'adresse `x-4` et on donne la valeur `0xABCDEF01` à cette adresse ainsi on écrase la val_2 qui n'avait pas de encore de sens à l'instant T car au dessus de la pile

=> `eax` vaut `0x01234567` et `esp` contient tjr la valeur `x-4`

=> mtn `esp` vaut la valeur `x-8` qui contient la valeur `0x01234567` à cette adresse

=> `esp` vaut `x-4` et la valeur à `x-8` n'a plus d'importance car au dessus de la tête de pile (même si elle a eu de l'importance à un moment) et `eax` prends la val `0xABCDEF01`

=> enfin `esp` vaut x qui contient val_3 à l'adresse x et au dessus ça n'a plus d'importance mais les vals ne changent pas donc `eax` vaut val_3

Exercice 2.

Donner des suites d'instructions utilisant uniquement les instructions `mov` et `add` afin de simuler l'instruction.

```
push eax
```

Faire la même chose pour l'instruction.

```
pop ebx
```

=> pour `push eax` :

```
add esp, -4
```

```
mov [esp], eax
```

```
sub esp, 4 ; On fait de la place sur la pile (esp ↘)
```

`mov [esp], eax` ; On copie eax au sommet de la pile

=> pour `pop ebx` :

`mov ebx, [esp]`

`add esp, 4`

`mov ebx, [esp]` ; On lit la valeur au sommet de pile

`add esp, 4` ; On "retire" ce sommet de la pile

Exercice 3.

Dans le code ci-contre, le registre `ebp` est utilisé pour sauvegarder la valeur de `esp`. Nous verrons plus tard que c'est en fait le rôle traditionnel de `ebp`.

Donner l'état des registres `eax`, `ebx`, `esp` et de la pile aux étiquettes un, deux, trois et quatre.

L'état de la pile doit être donné relativement à la valeur de `ebp`.

`mov ebp, esp` ; `ebp = x, esp = x`

`mov eax, 0` ; `eax = 0`

`mov ebx, 0` ; `ebx = 0`

`push dword 12` ; `[ebp-4] = 12, esp = ebp-4`

`push dword 13` ; `[ebp-8] = 13, esp = ebp-8`

`push dword 15` ; `[ebp-12] = 15, esp = ebp-12`

un :

`pop eax` ; `eax = 15, esp = ebp-8`

`pop ebx` ; `ebx = 13, esp = ebp-4`

`add eax, ebx` ; `eax = 15 + 13 = 28`

deux :

`push eax` ; `[ebp-8] = 28, esp = ebp-8`

`push ebx` ; `[ebp-12] = 13, esp = ebp-12`

`mov dword [esp + 8], 9` ; `[esp+8] = [ebp-4] = 9` (écrase le 12)

trois :

`pop eax` ; `eax = 13, esp = ebp-8`

`pop ebx` ; `ebx = 28, esp = ebp-4`

`pop ebx` ; `ebx = 9, esp = ebp`

quatre :

Après les trois `push` :

- `ESP = x-12`
- Pile (en descendant depuis `EBP`) :
 - `[x-4] = 12`
 - `[x-8] = 13`
 - `[x-12] = 15`
- `EAX = EBX = 0`

Etiquette 1 :

- `EAX = 28`

- EBX = 13
- ESP = x-4
- Pile : [x-4] = 12

Etiquette 2 :

- ESP = x-12
- Pile :
 - [x-12] = 13 <-- sommet !
 - [x-8] = 28
 - [x-4] = 9 (modifié ici !)
- EAX = 28
- EBX = 13

Etiquette 3 :

- ESP = x (pile vide)
- EAX = 13
- EBX = 9
- Pile vide

Etiquette 4 :

- ESP = x (pile vide)
- EAX = 13
- EBX = 9

Exercice 4.

Écrire un programme E4.asm qui lit des entiers au clavier tant qu'ils sont différents de -1, et affiche la liste des entiers lus dans l'ordre inverse. Par exemple, sur l'entrée 2 5 6 7 1 3 2 4 -1, le programme affichera 4 2 3 1 7 6 5 2.

Astuce 1. Il est possible (et recommandé) d'utiliser la pile pour enregistrer les entiers lus.

```
%include "asm_io.inc"
```

```
SECTION .data
```

```
msg_entree db "Entrez un entier (finir par -1) : ", 0
```

```
SECTION .bss
```

```
SECTION .text
```

```
global main
```

```
main:
```

```
    ; Saisie et empilement
```

```
lire:
```

```
    mov eax, msg_entree
```

```
    call print_string
```

```
    call read_int
```

```
cmp eax, -1
je depile
push eax
jmp lire
```

depile:

; À ce stade la pile contient tous les nombres lus, le dernier au sommet

affiche:

; On ne sait pas à l'avance combien il y en a, il faut donc tester la pile !
; Si la pile était vide au départ, le retour d'entrée -1 peut servir comme stop

; Simuler la dépilement : tant que la pile n'est pas vide
; Si tu as une convention sur l'entrée/sortie tu peux stocker esp
; Mais dans un contrôle d'exercice, il suffit d'un while

; On suppose ici qu'il reste au moins les valeurs entrées

suite_affichage:

```
cmp esp, [stack_start] ; Optionnel, si tu as stocké la valeur de départ pour repérer le
stop
je fin
pop eax
call print_int
call print_espace
jmp suite_affichage
```

fin:

```
call print_nl
mov ebx, 0
mov eax, 1
int 0x80
```

Explication pédagogique

- La pile stocke les entiers dans l'ordre de lecture.
- Par le comportement naturel de la pile (LIFO), la sortie lors du dépilement est dans l'ordre inverse de l'entrée—exactement ce que demande l'exercice.
- Les instructions push et pop sont spécialement dédiées à cet usage sur architectures x86, c'est leur usage classique.

Exercice 5.

Réaliser un programme E5.asm qui lit une suite d'entiers terminée par -1 et qui affiche cette liste triée dans l'ordre croissant. Un simple tri à bulles ou un tri par sélection suffira.

E5_BUBBLE.asm - Tri à bulles

```
%include "asm_io.inc"
```

SECTION .data

```
prompt db "Entrez une suite d'entiers terminee par -1 : ", 0
```

```
SECTION .bss
```

```
n resd 1
```

```
SECTION .text
```

```
global main
```

```
main:
```

```
    mov eax, prompt
```

```
    call print_string
```

```
    mov ecx, 0          ; n = 0
```

```
lire:
```

```
    call read_int
```

```
    cmp eax, -1
```

```
    je fin_lecture
```

```
    push eax
```

```
    inc ecx
```

```
    jmp lire
```

```
fin_lecture:
```

```
    mov [n], ecx        ; sauvegarde n
```

```
    mov ecx, [n]
```

```
    dec ecx             ; ecx = n-1 (passes)
```

```
boucle_passes:
```

```
    cmp ecx, 0
```

```
    jle affichage
```

```
    mov ebx, 0          ; j = 0
```

```
    mov eax, [n]
```

```
    sub eax, ecx
```

```
    mov edx, eax        ; edx = n - i (limite de j)
```

```
boucle_j:
```

```
    cmp ebx, edx
```

```
    jge fin_passe
```

```
    ; charger a[j] et a[j+1] depuis la pile
```

```
    mov eax, [esp + ebx*4]    ; a[j]
```

```
    mov esi, [esp + ebx*4 + 4] ; a[j+1]
```

```
    cmp eax, esi
```

```
    jle pas_echanger
```

```
    ; echanger a[j] et a[j+1]
```

```
    mov [esp + ebx*4], esi
```

```
    mov [esp + ebx*4 + 4], eax
```

```

pas_echanger:
    inc ebx
    jmp boucle_j

fin_passe:
    dec ecx
    jmp boucle_passes

affichage:
    mov ecx, [n]
    mov ebx, 0

aff_loop:
    cmp ebx, ecx
    jge fin_prog

    mov eax, [esp + ebx*4]
    call print_int
    call print_espace

    inc ebx
    jmp aff_loop

fin_prog:
    call print_nl
    mov ebx, 0
    mov eax, 1
    int 0x80

```

E5_SELECTION.asm - Tri par sélection

```
%include "asm_io.inc"
```

```
SECTION .data
```

```
prompt db "Entrez une suite d'entiers terminee par -1 : ", 0
```

```
SECTION .bss
```

```
n resd 1
```

```
SECTION .text
```

```
global main
```

```
main:
```

```
    mov eax, prompt
```

```
    call print_string
```

```
    mov ecx, 0          ; n = 0
```

```
lire:
```

```
call read_int
cmp eax, -1
je fin_lecture
push eax
inc ecx
jmp lire
```

```
fin_lecture:
    mov [n], ecx        ; sauvegarde n

    mov ecx, 0          ; i = 0
```

```
boucle_i:
    cmp ecx, [n]
    jge affichage

    mov ebx, ecx        ; min = i
    mov edx, ecx
    inc edx             ; j = i+1
```

```
boucle_j:
    cmp edx, [n]
    jge fin_i

    ; charger a[j] et a[min]
    mov eax, [esp + edx*4] ; a[j]
    mov esi, [esp + ebx*4] ; a[min]

    cmp eax, esi
    jge pas_min

    ; a[j] < a[min], donc min = j
    mov ebx, edx
```

```
pas_min:
    inc edx
    jmp boucle_j
```

```
fin_i:
    ; echanger a[i] et a[min]
    mov eax, [esp + ecx*4] ; a[i]
    mov esi, [esp + ebx*4] ; a[min]

    mov [esp + ecx*4], esi
    mov [esp + ebx*4], eax

    inc ecx
    jmp boucle_i
```

```

affichage:
    mov ecx, 0

aff_loop:
    cmp ecx, [n]
    jge fin_prog

    mov eax, [esp + ecx*4]
    call print_int
    call print_espace

    inc ecx
    jmp aff_loop

fin_prog:
    call print_nl
    mov ebx, 0
    mov eax, 1
    int 0x80

```

Exercice 6.

Pour le code ci-contre, donner une suite d'instructions n'utilisant que push et jmp qui soit équivalente à l'instruction call dans l'instruction call print_int de la ligne 1.

```

    call print_int
suite :
    mov eax, 0
    ...

print_int :
    ...

```

Principe technique :

L'instruction call print_int fait :

- empile l'adresse de l'instruction suivante (« retour »),
- saute à print_int.

Pour simuler ça :

- Empiler l'adresse de retour (l'instruction après le call)
- Sauter à la fonction (jmp print_int)

Code équivalent :

```

push suite      ; suite est le label de l'instruction après l'appel
jmp print_int

```

suite:

```
mov eax, 0
```

```
...
```

Dans la fonction : ret dépile l'adresse et y saute, ce qui revient à "retour d'appel".

Exercice 7.

Donner l'ordre dans lequel sont exécutées les instructions ci-contre.

Pour cela, mentionner pour chaque étape de l'exécution, le numéro de la ligne exécutée ainsi que l'état de la pile si celle-ci a été modifiée par l'instruction en question.

main :

```
call f ; Ligne 1
```

l1 :

```
call g ; Ligne 2
```

l2 :

```
mov ebx, 0 ; Ligne 3
```

```
mov eax, 1 ; Ligne 4
```

```
int 0x80 ; Ligne 5
```

f :

```
call g ; Ligne 6
```

l3 :

```
ret ; Ligne 7
```

g :

```
ret ; Ligne 8
```

Étapes d'exécution (en numérotant les lignes):

1. Ligne 1 : call f :
 - Empile l'adresse de l1
 - Saut à f
 - Pile : [l1]
2. Ligne 6 : call g :
 - Empile l'adresse de l3
 - Saut à g
 - Pile : [l3][l1]
3. Ligne 8 : ret (de g)
 - Dépile l3, saute à l3 (ligne 7)
 - Pile : [l1]
4. Ligne 7 : ret (fin de f)

- Dépile I1, saute à I1 (ligne 2)
- Pile vide
- 5. Ligne 2 : call g
 - Empile I2
 - Saut à g
 - Pile : [I2]
- 6. Ligne 8 : ret (de g)
 - Dépile I2, saute à I2 (ligne 3)
 - Pile vide
- 7. Ligne 3, 4, 5 : Le programme termine

Exercice 8.

Expliquer si la suite d'instructions ci-contre est correcte.
Dans le cas contraire, expliquer pourquoi.

```
fonction :
    call read_int
    push eax
    pop ebx
    push ecx
    ret
```

Est-ce correct ?

- call read_int lit une valeur dans eax.
- push eax puis pop ebx : transfère le contenu de eax dans ebx.
- push ecx : stocke ecx sur la pile, mais n'est jamais dépilé !
- ret : dépile une valeur, qui devrait être une adresse de retour — mais ici, on a juste mis ecx en sommet de pile !

Problème :

Cette suite peut corrompre la pile : le dernier push ecx met la valeur de ecx à la place de l'adresse de retour attendue par ret, donc la fonction retournera vers une "adresse" prise dans ecx au lieu du vrai retour.

Résumé : Cette suite n'est pas correcte. Au retour de fonction, il faut que la pile contienne l'adresse de retour attendue

Exercice 9.

Expliquer le comportement des programmes Probleme1.asm et Probleme2.asm. On y trouve des instructions que nous n'avons pas rencontrées auparavant. Voici leur explication : pusha empile le contenu de tous les registres, c'est à dire eax, ebx, ecx, edx, esp, ebp, esi et edi. Cette instruction est utile pour effectuer une sauvegarde générale des valeurs des registres avant de les modifier ; popa restaure les valeurs de tous les registres dont les valeurs ont été enregistrées au préalable dans la pile par l'intermédiaire de pusha ;

Probleme1.asm (rappel) :

- affiche fait push dword affiche (adresse de la fonction sur pile, sans usage) avant de faire ses affichages puis ret.
- Cela pollue la pile et n'a aucun intérêt ici.

Probleme2.asm :

- Sauvegarde tous les registres avec pusha, puis les restaure avec popa après l'appel à une fonction.
- Utilise aussi ebp comme repère de "cadre de pile" et remet esp à sa valeur initiale en fin de fonction.
- Utile pour restaurer l'état global des registres, recommandé si tes fonctions modifient beaucoup de registres.

En résumé :

- pusha/popa permettent la sauvegarde/restauration massive et rapide de tous les registres de travail (sauf les registres segment, et stack pointer lui-même), pour ne pas "polluer" l'état du programme principal.

Exercice 10.

Réécrire la fonction print_string en utilisant l'appel système write. Faire en sorte qu'après l'exécution de la fonction, les registres soient dans le même état qu'avant l'exécution de la fonction. La fonction prend en argument une adresse d'une chaîne de caractères qui se termine par le caractère nul (de code ASCII 0, encore appelé marqueur de fin de chaîne). La fonction se nommera print_string2 et sera écrite dans un fichier E10.asm.

Principe :

- Appel système Linux :
 - eax = 4 (write)
 - ebx = 1 (stdout)
 - ecx = adresse de la chaîne
 - edx = longueur de la chaîne (calculée au préalable, car la chaîne finit par 0)
- Sauvegarder/restaurer tous les registres modifiés (ex: pusha / popa)
- Calculer la longueur de la chaîne en parcourant jusqu'au caractère nul.

print_string2:

```
pusha
mov esi, [esp + 36] ; Paramètre: adresse de la chaîne (si passée par push avant call)
mov ecx, esi
xor edx, edx
```

.longueur:

```
cmp byte [ecx], 0
je .suite
inc ecx
inc edx
jmp .longueur
```

.suite:

```
mov eax, 4 ; syscall write
mov ebx, 1 ; descriptor "stdout"
```

```

mov ecx, esi
int 0x80
popa
ret

```

Exercice 11.

Écrire une fonction longueur qui respecte les conventions d'appel du C et qui prend en argument l'adresse d'une chaîne de caractères terminée par un octet et calcule sa longueur.

```

longueur:
    push ebp
    mov ebp, esp
    push ebx          ; sauvegarde EBX

    ; Adresse de la chaine
    mov ebx, [ebp+8]
    ; Init. compteur
    xor eax, eax

    ; Boucle de comptage
boucle:
    cmp byte [ebx], 0
    je fin_boucle

    inc eax            ; incremente compteur
    add ebx, 1         ; pointe au caractere suivant
    jmp boucle

fin_boucle:
    ; Restauration des registres
    pop ebx
    pop ebp
    ret               ; EAX = longueur

```

Exercice 12.

Réécrire la fonction print_string2 (dans un nouveau fichier) en suivant la convention d'appel du C. La fonction devra utiliser la fonction longueur précédente.

```

print_string2:
    push ebp
    mov ebp, esp
    ; Sauvegarde des registres
    push ebx
    push ecx
    push edx

    ; Adresse de la chaine
    mov ebx, [ebp+8]

```

```

; Appel a longueur pour obtenir la longueur
push ebx
call longueur
add esp, 4

; EAX = longueur, on la met dans EDX
mov edx, eax

; Preparation du syscall write
mov ecx, ebx      ; ECX = adresse de la chaine
mov eax, 4        ; syscall write
mov ebx, 1        ; stdout
int 0x80

; Restauration des registres
pop edx
pop ecx
pop ebx
pop ebp
ret

```

EXEMPLE DE MAIN :

SECTION .data

```
ma_chaine db "Bravo, c'est cdecl!", 10, 0
```

SECTION .text

global main

main:

```

push ma_chaine
call print_string2
add esp, 4

```

```

mov eax, 1
mov ebx, 0
int 0x80

```